

BioWatchCare

Documentation claire du smart contract Solana / Anchor

| **Programme ID** : 24paEKyzz4aoBDmpdGVyC4Larnp7Bwka95TkgLHoLTiC

| **Framework** : Anchor sur Solana

| **Version documentée** : état actuel du dépôt au 9 mars 2026

| **But métier** : fournir une base on-chain pour un carnet de santé numérique, avec
| consentement, prescriptions, dispensation, facturation, remboursement et audit.

1. Résumé exécutif

Ce smart contract ne stocke pas les données médicales lisibles en clair. Il stocke surtout des **preuves**, des **hashes**, des **permissions**, des **statuts**, des **liens logiques** entre acteurs et patients, ainsi qu'un **journal d'audit**.

C'est la bonne base pour une architecture où l'utilisateur final se connecte avec **email + mot de passe** dans une application classique, pendant que la blockchain fonctionne en arrière-plan via un backend custodial.

Idée centrale : la blockchain sert ici de registre de confiance. Elle ne remplace ni la base de

données applicative, ni le stockage chiffré des dossiers médicaux.

2. Ce que fait le contrat

- Initialiser la configuration globale du programme.
- Gérer les rôles des entités autorisées : hôpital, assureur, médecin, pharmacien.
- Créer un profil patient pseudonymisé via un hash d'identifiant.
- Gérer les consentements d'accès à certaines catégories de données.
- Ancrer des prescriptions médicales sous forme de hash et pointeur hors chaîne.

- Émettre et vérifier des QR tokens pour la dispensation pharmaceutique.
- Créer des factures et traiter des demandes de remboursement.
- Enregistrer des identités custodiales liées à des utilisateurs applicatifs.
- Ancrer des documents médicaux génériques.
- Journaliser les accès à des ressources médicales.

3. Ce que le contrat ne doit pas faire

- Stocker les noms, adresses, numéros de téléphone ou emails des patients en clair.
- Stocker les comptes rendus médicaux complets, PDF, images ou analyses détaillées.
- Gérer l'authentification email / mot de passe.
- Remplacer la logique applicative de chiffrement, de session ou de récupération de compte.

4. Architecture recommandée autour de ce contrat

Couche	Responsabilité	Exemples
Frontend	Interface utilisateur, formulaires, consultation, workflow produit.	Connexion email, tableau de bord patient, consultation de dossier.
Backend	Authentification, wallets custodiaux, chiffrement, stockage métier, envoi des transactions Solana.	Node/Express, Postgres, stockage objet chiffré, KMS/Vault.
Blockchain	Preuve, audit, consentement, statuts, ancrage de documents et traçabilité.	PDAs Anchor, événements, rôles et journaux d'accès.

Frontend Web / Mobile Connexion email + mot de passe
Formulaires patient / médecin

Consultation des dossiers Aucune exposition du wallet Backend Custodial Auth, sessions,

rôles applicatifs Wallets gérés par la plateforme Chiffrement et stockage off-chain

Construction et envoi des transactions Audit et orchestration métier Solana / Anchor Rôles et

consentements Hashes et ancres médicales QR, factures, claims Journal d'accès Base métier

+ stockage chiffré hors chaîne

Diagramme d'architecture cible : le frontend parle au backend, et le backend pilote la blockchain

sans imposer de wallet à l'utilisateur final.

5. Objets on-chain et rôle de chacun

5.1 GlobalConfig

Compte global du programme. Il contient l'administrateur principal et le seuil de remboursement automatique.

Champ	Type	Utilité
admin	Pubkey	Autorité centrale du programme.
auto_reimb_threshold	u64	Montant maximal auto-approuvé pour un claim.
bump	u8	Bump PDA Anchor.
created_at	i64	Date de création.

5.2 EntityRole

Répertoire des entités métier autorisées. Une entité peut être Hospital, Insurer, Doctor ou Pharmacist.

5.3 PatientProfile

Représente un patient de manière pseudonymisée. Le patient n'est pas identifié par son nom, mais

par un `patient_id_hash`.

5.4 Consent

Autorisation accordée à un tiers pour consulter certaines catégories d'informations du patient. Le consentement inclut un **bitmask de scopes**, une date d'expiration éventuelle et un indicateur de révocation.

5.5 Prescription

Ancre une ordonnance créée par un médecin approuvé. Le contenu réel reste hors chaîne, le contrat conserve seulement le hash et un pointeur hashé.

5.6 QrToken

Jeton temporaire, valable 48 heures, utilisé pour sécuriser la délivrance d'une ordonnance en pharmacie.

5.7 Dispense

Trace la dispensation effective d'un médicament après utilisation d'un QR token valide.

5.8 Invoice

Enregistre une facture associée à un patient, avec son montant et sa devise.

5.9 ClaimStatus

Suit la décision de remboursement prise par l'assureur. Le claim peut être auto-approuvé, en attente, approuvé ou rejeté.

5.10 UserIdentity

Nouveau compte ajouté pour le modèle custodial. Il relie un utilisateur applicatif hors chaîne à un wallet Solana contrôlé par votre plateforme.

Champ	Type	Utilité
<code>app_user_id_hash</code>	[u8; 32]	Hash de l'identifiant utilisateur applicatif.

Champ	Type	Utilité
wallet	Pubkey	Wallet custodial associé.
role_class	Enum	Classe métier globale : patient, practitioner, operator.
status	Enum	Active ou Suspended.

5.11 MedicalRecordAnchor

Nouveau compte générique pour ancrer n'importe quel document de santé : consultation, labo, imagerie, vaccination, note clinique, document externe.

5.12 AccessEvent

Nouveau compte d'audit. Il journalise un accès à une ressource médicale pour un patient donné. Il conserve qui a accédé, à quoi, de quelle manière, et quand.

6. Énumérations importantes

Enum	Valeurs	Rôle
Role	Hospital, Insurer, Doctor, Pharmacist	Rôles métier des entités.
EntityStatus	Pending, Approved, Revoked	Cycle de vie d'une entité.
UserRoleClass	Patient, Practitioner, Operator	Classe large de l'identité applicative.
IdentityStatus	Active, Suspended	Activation d'une identité custodiale.
RecordType	Consultation, LabResult, Imaging, Vaccination, ClinicalNote, ExternalDocument	Nature du document médical ancré.
RecordStatus	Active, Archived, Superseded	État du document médical.
AccessKind	Read, Write, Share, Export	Type d'accès journalisé.

Enum	Valeurs	Rôle
<code>ClaimDecision</code>	AutoApproved, Pending, Approved, Rejected	Décision assureur.

7. Permissions et scopes

Le contrat définit trois scopes de consentement :

- `VIEW_RECORDS` : accès aux dossiers médicaux.
- `VIEW_PRESCRIPTIONS` : accès aux ordonnances.
- `VIEW_INVOICES` : accès aux factures.

`VIEW_ALL` combine les trois. Le consentement est stocké comme un bitmask, ce qui permet d'exprimer plusieurs permissions dans une seule valeur.

8. Fonctions du smart contract

8.1 Administration

`initialize_config(threshold: Option<u64>)`

But : créer la configuration globale du programme.

Utilisation : à exécuter une seule fois au déploiement.

Effet : crée le PDA `GlobalConfig` et fixe l'admin.

`set_threshold(new_threshold: u64)`

But : modifier le seuil d'auto-remboursement.

Utilité métier : automatiser les petits remboursements et laisser les gros montants en revue manuelle.

`register_entity(role, entity_pubkey, metadata_hash)`

But : enregistrer une entité métier avec statut initial `Pending`.

Exemple : enregistrer un médecin ou un hôpital avant validation finale par l'administrateur.

`approve_entity()`

But : approuver une entité et l'autoriser à agir dans le système.

`revoke_entity()`

But : révoquer une entité.

Effet : l'entité reste référencée on-chain mais ne peut plus effectuer d'actions protégées.

8.2 Gestion patient et consentement

`create_patient_profile(patient_id_hash)`

But : créer le profil pseudonymisé d'un patient.

Qui peut le faire : un hôpital ou un assureur approuvé.

`grant_consent(patient_id_hash, grantee_pubkey, scopes, expires_at)`

But : donner l'autorisation à un tiers d'accéder à certaines catégories de données.

Point clé : le patient signe, mais dans un modèle custodial cette signature peut être émise par votre backend au nom du patient.

`revoke_consent(patient_id_hash, grantee_pubkey)`

But : retirer un consentement existant.

8.3 Parcours prescription et pharmacie

`add_prescription(patient_id_hash, rx_hash, pointer_hash)`

But : ancrer une ordonnance pour un patient.

Données stockées : uniquement le hash de l'ordonnance et un hash de pointeur vers le stockage off-chain.

`cancel_prescription()`

But : annuler une ordonnance active.

`issue_qr_token(token_hash)`

But : créer un QR token expirant au bout de 48 heures.

`verify_qr_token()`

But : vérifier qu'un QR token est toujours valide sans le consommer.

`dispense_with_qr(dispense_hash)`

But : enregistrer la dispensation d'un médicament et marquer le QR token comme utilisé.

8.4 Facturation et remboursement

`create_invoice(patient_id_hash, invoice_hash, amount, currency_code)`

But : créer une facture rattachée à un patient.

`auto_or_pending_claim(insurer)`

But : ouvrir une demande de remboursement.

Si le montant de la facture est inférieur ou égal au seuil global, le claim est immédiatement AutoApproved. Sinon, il passe à Pending.

`insurer_decide_claim(approve, reason_code)`

But : permettre à l'assureur de statuer sur un claim en attente.

Comportement : le code motif n'est utilisé qu'en cas de rejet.

8.5 Nouvelles fonctions pour le modèle carnet de santé custodial

`register_user_identity(app_user_id_hash, wallet, role_class)`

Associe un utilisateur applicatif à un wallet géré par la plateforme. C'est l'un des pivots du modèle où l'utilisateur final se connecte avec email et mot de passe sans manipuler de wallet.

`add_medical_record_anchor(patient_id_hash, record_type, record_hash, pointer_hash, version)`

Ajoute une ancre générique de document médical. Cela permet de gérer plus qu'une simple ordonnance : consultation, résultats, documents externes, etc.

`supersede_medical_record(superseded_by)`

Marque un document médical comme remplacé par une nouvelle version ou un nouvel enregistrement. Très utile quand un document a été corrigé ou mis à jour.

`log_access_event(patient_id_hash, access_kind, resource_hash)`

Journalise un accès à une ressource médicale. L'événement est dérivé par `patient + accessor + resource_hash`.

Cela permet de prouver qu'un acteur donné a consulté, modifié, partagé ou exporté une ressource liée à un patient.

9. Schémas visuels des flux

9.1 Flux global dossier médical

Patient consentement Médecin / Hôpital création dossier / prescription Backend chiffre et

signe Solana hash + audit Storage données chiffrées 1. autorise / consulte 2. prépare

l'opération 3. ancre la preuve 4. conserve le contenu

Flux cible : les données complètes restent hors chaîne, la blockchain conserve la preuve, le contrôle

d'accès et la traçabilité.

9.2 Parcours patient

Créer compte Profil patient hashé on-chain Consentement grant / revoke Consulter dossier

hors chaîne Historique audit d'accès

Le patient ne manipule pas la blockchain directement. Il gère son accès et consulte son dossier

comme dans une application classique.

9.3 Parcours médecin

Rôle approuvé Doctor / Hospital Consulter patient si consentement Créer record ou

prescription Ancrer hash sur Solana Mettre à jour ou remplacer

Le médecin alimente le carnet via des documents hashés et des prescriptions, avec traçabilité

complète.

9.4 Parcours pharmacien

Rôle pharmacien approuvé Scanner QR token Vérifier validité non expiré / non utilisé

Dispense marque le token utilisé Audit trace finale

Le pharmacien vérifie le QR token avant dispensation, puis la consommation du token empêche les

usages multiples.

9.5 Parcours assureur

Rôle assureur approuvé Recevoir claim auto ou pending Analyser politique métier Approuver

ou rejeter Tracer la décision

Le claim suit un cycle simple : auto-approbation sous seuil, sinon décision assureur avec motif

éventuel de rejet.

10. PDAs utilisés

Compte	Seeds
GlobalConfig	["config"]
UserIdentity	["user_identity", app_user_id_hash]
EntityRole	["role", entity_pubkey]
PatientProfile	["patient", patient_id_hash]
Consent	["consent", patient_pda, grantee_pubkey]
MedicalRecordAnchor	["record", patient_pda, record_hash]
Prescription	["rx", patient_pda, rx_hash]
QrToken	["qr", prescription_pda, token_hash]
Dispense	["dispense", prescription_pda, pharmacist_pubkey]
Invoice	["invoice", patient_pda, invoice_hash]
ClaimStatus	["claim", invoice_pda, insurer_pubkey]
AccessEvent	["access_event", patient_pda, accessor_pubkey, resource_hash]

11. Événements émis

Chaque événement Anchor permet de suivre l'activité on-chain sans relire tous les comptes à chaque fois. Les événements principaux sont :

- `ConfigInitialized`, `ThresholdUpdated`
- `EntityRegistered`, `EntityApproved`, `EntityRevoked`
- `PatientCreated`
- `ConsentGranted`, `ConsentRevoked`
- `PrescriptionCreated`, `PrescriptionCancelled`
- `QrIssued`, `QrVerified`, `QrUsed`
- `Dispensed`
- `InvoiceCreated`, `ClaimAutoApproved`, `ClaimPending`, `ClaimApproved`, `ClaimRejected`
- `UserIdentityRegistered`, `MedicalRecordAnchored`, `MedicalRecordSuperseded`, `AccessEventLogged`

12. Gestion des erreurs

Le contrat définit plusieurs erreurs métier explicites :

Erreur	Signification
<code>NotAdmin</code>	L'appelant n'est pas l'administrateur configuré.
<code>Unauthorized</code>	L'acteur signataire n'est pas autorisé pour cette action.
<code>RoleNotApproved</code>	Le rôle métier existe mais n'est pas approuvé.
<code>InvalidRole</code>	Le rôle métier ne correspond pas à l'instruction.
<code>ConsentMissing</code>	Le consentement attendu n'existe pas.
<code>ConsentExpired</code>	Le consentement est expiré.
<code>ConsentRevoked</code>	Le consentement a été révoqué.
<code>InvalidScopes</code>	Le bitmask de scopes est invalide.
<code>RxCancelled</code>	L'ordonnance est déjà annulée ou non exploitable.

Erreur	Signification
<code>QrExpired</code>	Le QR token est expiré.
<code>QrAlreadyUsed</code>	Le QR token a déjà été consommé.
<code>InvalidCurrencyCode</code>	Le code devise n'est pas valide.
<code>ClaimAlreadyDecided</code>	Le claim n'est plus en attente.
<code>IdentityInactive</code>	L'identité applicative liée au wallet n'est pas active.
<code>RecordAlreadySuperseded</code>	Le document médical est déjà remplacé ou non actif.

13. Répartition recommandée des données

On-chain

Hash patient, hash documents, consentements, rôles, statuts, claims, timestamps, journal d'accès, pointeurs hashés.

Off-chain

Emails, mots de passe, identité civile, documents médicaux lisibles, PDF, imagerie, comptes rendus, pièces justificatives, sessions utilisateur.

14. Lecture métier du contrat

1. L'admin initialise le système et approuve les acteurs.
2. Un hôpital ou un assureur crée un profil patient pseudonymisé.
3. Le patient accorde ou retire des consentements.
4. Un médecin crée des ordonnances ou des ancres de documents médicaux.
5. Un pharmacien vérifie puis consomme un QR token pour délivrer un traitement.
6. Un hôpital crée une facture.
7. Le système auto-approuve un claim si le montant est faible, sinon l'assureur tranche.
8. Chaque accès important peut être journalisé via `AccessEvent`.

15. Forces du design actuel

- Bonne séparation entre preuve on-chain et données sensibles off-chain.
- Modèle de rôles clair.
- Pseudonymisation native des patients.
- Workflow prescription → QR → dispensation cohérent.
- Nouveau socle custodial compatible avec email + mot de passe côté produit.
- Audit des accès désormais possible.

16. Limites actuelles à connaître

- Le contrat ne gère pas lui-même l'authentification applicative.
- Le chiffrement des dossiers médicaux doit être fait hors chaîne.
- Le contrat ne stocke pas encore une liste explicite de versions reliées entre documents, seulement un champ `superseded_by`.
- Le modèle d'audit actuel crée un événement unique par combinaison `patient + accessor + resource_hash`. Si vous voulez plusieurs lectures identiques historisées séparément, il faudra prévoir une clé plus riche.

17. Recommandation produit

Pour votre MVP, il faut conserver ce smart contract comme **registre de confiance**, puis construire autour :

- une API backend avec authentification email / mot de passe,
- des wallets custodiaux signant les transactions à la place des utilisateurs,
- une base de données métier,
- un stockage chiffré des documents médicaux,
- une interface web centrée sur les dossiers et les workflows, pas sur la blockchain.

Conclusion : le smart contract actuel est désormais adapté à une architecture de carnet de

santé numérique où la blockchain reste invisible pour l'utilisateur final, tout en apportant

preuve, traçabilité, contrôle d'accès et audit.

| Annexe A - Pour les développeurs

18. Annexe technique développeurs

Cette annexe vise l'équipe technique qui doit intégrer, maintenir ou faire évoluer le système.

18.1 Points d'intégration backend

- Le backend doit être le fee payer et l'orchestrateur des signatures.
- Les wallets patients peuvent être custodiaux et liés à `UserIdentity`.
- Les documents médicaux doivent être chiffrés avant stockage off-chain.
- Le backend doit conserver la table de correspondance entre identités applicatives, wallets custodiaux, patients et rôles métier.

18.2 Recommandations d'implémentation

- Auth : email + mot de passe avec Argon2, session sécurisée et MFA pour les professionnels.
- Clés : KMS, Vault ou HSM pour isoler les wallets custodiaux.
- Stockage : base SQL pour le métier, stockage objet chiffré pour les fichiers.
- Hashing : SHA-256 homogène côté backend pour documents, pointeurs et identifiants pseudonymisés.
- Audit : persister aussi un journal off-chain pour les événements applicatifs non on-chain.

18.3 Évolutions naturelles du contrat

- ajouter un vrai historique multi-événements pour des lectures répétées d'une même ressource,
- ajouter des types de documents supplémentaires,
- lier explicitement une chaîne de versions entre documents,
- ajouter des politiques d'accès plus fines que le simple bitmask actuel.

18.4 Vérification actuelle

La suite locale valide actuellement :

- configuration globale et seuil,
- cycle de vie des rôles,
- patient et consentement,
- prescription, QR et dispensation,
- facturation et claims,
- identité custodiale, ancrage médical et audit d'accès.

■ Annexe B - Pour les investisseurs

19. Annexe synthèse investisseurs

Cette annexe traduit la valeur produit sans entrer dans les détails d'implémentation.

19.1 Problème adressé

Les données de santé sont sensibles, fragmentées et souvent difficiles à partager avec preuve d'intégrité. Les utilisateurs finaux ne veulent pas gérer un wallet crypto ni payer des frais blockchain.

19.2 Réponse de BioWatchCare

- expérience utilisateur simple, type application santé classique,
- traçabilité forte des accès et actions critiques,
- preuve d'intégrité des documents médicaux,
- contrôle de consentement patient,
- automatisation partielle du remboursement.

19.3 Avantage stratégique

- la blockchain est invisible côté utilisateur,
- les coûts restent maîtrisés car seules les preuves sont on-chain,
- l'architecture est compatible avec des exigences fortes d'audit,
- le socle est extensible vers assurance, pharmacie, hôpital et télémédecine.

19.4 Lecture business

BioWatchCare peut être présenté comme un registre de confiance pour l'écosystème santé, avec une logique SaaS / B2B2C : établissements de santé, réseaux de soins, assureurs et patients.

Document généré à partir du code source actuel du programme Anchor BioWatchCare.